

Project Report

Leisha Garg, Ritsika Medury, Bavya Sri Sai Inti, Rikhita Baddula

1. Please list out changes in the directions of your project if the final project is different from your original proposal (based on your stage 1 proposal submission).

We removed the major-specific degree requirement aspect of our project since we were unable to scrape requirements for different majors due to inconsistent structure. Due to this we had to remove our creative component that allowed users to filter courses that meet a degree requirement based on major pre-requisites. We also added a visual to show each instructor's average GPA for the specific course.

2. Discuss what you think your application achieved or failed to achieve regarding its usefulness.

Our application was successful in displaying all the CS courses along with its instructors and their average GPAs for students to understand the difficulty of the course. It was also successful in displaying the prerequisites for each course. Students can see multiple prerequisite options and the respective courses to their degree planning.

We were unable to scrape the requirements for each major since the format was different across majors and not consistent. The application could not provide major-specific degree requirements which makes it hard for students to view their full graduation progress rather they could see prerequisites and concurrent enrollment for a class they plan to take.

3. Discuss if you changed the schema or source of the data for your application

We did not change where our main data came from. We used the Course Catalog Data, Gened Dataset, GPA Dataset WAF datasets for our Course Information, Geneds Dataset, Course GPA by Instructor table. Since prerequisite and concurrent enrollment information was not available in a dataset, we scraped and processed that data ourselves. We also kept the same relational schema that we originally designed. The only changes we made were adding tables to store the scraped prerequisite and

concurrent enrollment data so it could properly fit into our database and be used by the application.

4. Discuss what you change to your ER diagram and/or your table implementations. What are some differences between the original design and the final design? Why? What do you think is a more suitable design?

The major change we made to our UML diagram was simplifying our User Table. Initially we had username, first name, last name, graduation month and year, but we realised that some of these columns were redundant, for example, we realised that netid is unique for each student which makes username redundant. Hence, we only decided to use netid as the required identifier and we got rid of other personal fields like first name, last name, graduation month and year.

5. Discuss what functionalities you added or removed. Why?

We are not asking users to select their major, pending semesters, and check all the courses they have completed so far from a prepopulated list. Instead, we made it broader such that any user could add their completed courses even from highschool. We modified the functionality to be a typing box where the user can type the course they completed so it doesn't restrict them from choosing from a specific list of courses. We also removed the filter functionality as we realized that users don't necessarily pick random courses that have the highest GPA but rather the courses they are interested in. However they do prefer to pick instructors with higher GPA average for a certain course which is why we added a visualization that sorts instructor GPA averages allowing users to view all the instructors and their GPAs in an intuitive manner.

6. Explain how you think your advanced database programs complement your application.

One of the advanced database programs we used was a trigger and this complemented our application by removing a course from student's planned courses as soon as they marked it complete which avoids confusion for the user so that a course cannot be marked planned and completed at the same time. This trigger automatically updated the database allowing users to not worry about manually syncing their courses.

Another advanced database program we use was transaction to allow users to add a course to their semester plan. This complemented our application by

checking multiple rules at once. For example, has the user already planned the course, whether adding this course would exceed the credit limit and whether requirements are satisfied. This transaction checks all these at once which makes the application more reliable and helps users to avoid invalid semester plans.

7. Each team member should describe one technical challenge that the team encountered. This should be sufficiently detailed such that another future team could use this as helpful advice if they were to start a similar project or where to maintain your project.

- 1) We ran out of GCP Billing Credits multiple times which forced us to learn what the different GCP roles and permissions are. We especially learnt about billing administrator and why the role needs to manage credits, enable billing and control project spending. This helped us understand how cloud permissions and cost-management work in the real-world.
- 2) One technical challenge we faced was that some of our advanced queries were not using up cost, that is our cost was the same before and after running our indexes. Through this we learnt that our optimizer was already using the most optimal plan.
- 3) Transactions: Another technical challenge we faced was learning how to use transactions correctly when updating the database. At first, we ran multiple SQL statements separately, and if one failed, the data could become inconsistent. We had to learn to group related updates into a single transaction so they would either all succeed or all fail together. Future teams should make sure to use transactions whenever multiple database changes depend on each other.
- 4) One of the major technical challenges that we encountered was extracting the prerequisite and concurrent enrollment information from the Course Explorer Dataset. The dataset did not store the prerequisites in a clean relational format. Instead each course had a long section info which is a string that can contain prerequisite or concurrent enrollment information. As this column is not standardized and many rows can also leave it blank or phrase the requirements differently we had to write a custom text matching and parsing logic to reliably extract the information efficiently. This turns the plain string text that is not standardized into a structured prerequisite and concurrent enrollment relationship such that we get a prerequisite or concurrent enrollment for each of the specified courses.

8. Are there other things that changed comparing the final application with the original proposal?

Yes one of the major changes from the original proposal to the final application is that we didn't implement a feature that we suggested we would. The feature is where a user clicks on a pending requirement and they are taken to a new

landing page where they would filter all courses by satisfying the requirement by GPA averages, see eligibility for new courses based on previous courses, and view semester based eligibility. Although in the final application users will still be able to view the requirements and plan courses but the level of detail with filterability eligibility was prohibited due to the scope and time constraints.

9. Describe future work that you think, other than the interface, that the application can improve on

Answer: In the future, we can take this further by modeling complex logical conditions such as the minimum grade constraints for courses that students are required to take before taking future courses. We could also plan to distinguish between the required and recommended courses within the given CS and CS + X tracks such that the system can give a path that the students can take. This accounts for the student's required courses and electives that would complement the student's interests. Specifically, if the students are interested in any particular tracks such as AI/ML or UI/UX we cater the student's plan such that we would suggest electives and course classes and would set them up to take the higher level classes in their specific specialization. This would make the planning logic for the student to be more smarter as the application not only checks for the eligibility aspect of it but would also let the students prioritize their courses based on what they intend to take and what they are taking right now.

10. Describe the final division of labor and how well you managed teamwork

Answer: We divided the work evenly across the team and coordinated closely throughout the project. For shared tasks such as setting up Google Cloud and configuring the databases, we met and worked together so everyone understood the system and could contribute effectively. We also held in-person meetings for key design decisions, such as finalizing the ER and UML diagrams, to ensure the entire team agreed on the tables and structure before moving forward. After aligning on the design, we split responsibilities for writing SQL and implementing different components, but continued meeting regularly during each stage to troubleshoot issues collaboratively and keep our progress synchronized.